DLD MILESTONE 2 REPORT

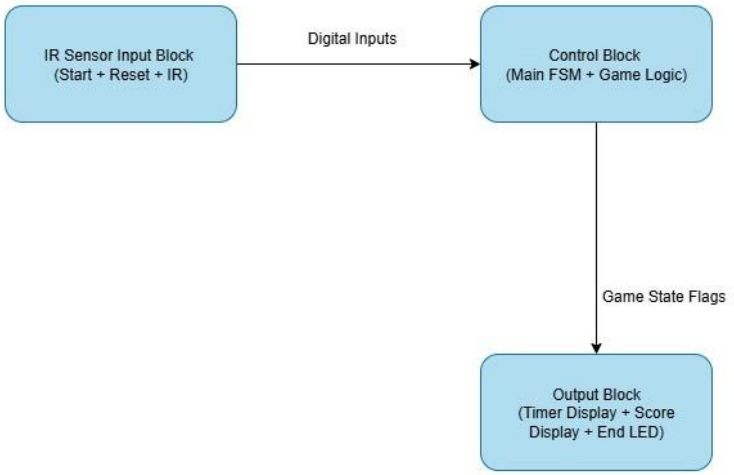
[Ayesha Mir (09886), Divya Lakhani (10267), Areesha Ashfaq (09879), Raheem Waseem Khan (09546)]

Introduction:

This project implements a Target shooting game like at carnivals called 'Point Blank'. It is implemented on the Basys-3 FPGA board. The primary objective of Milestone-2 was to complete the core gameplay logic in hardware, ensuring that all fundamental components of the system interact correctly in real time. This included designing and integrating the timer module, single sensor scoring system, debouncing circuits, and game-over logic, as well as validating the functionality through hardware testing.

In this stage, we focused on building the actual behavior of the game: the timer starts upon the user's input, IR sensor detect falling can through rising-edge transitions, the score increments accordingly, and the system transitions into a game-over state either when the timer reaches zero or the player achieves the maximum score. By the end of Milestone-2, the entire internal logic; inputs, control blocks, and outputs were operational and fully tested on the FPGA. This establishes the foundation for the next milestone, where the system will be able to handle multiple sensor inputs, visual display, and user interface elements will be implemented through VGA, and the whole game will come together.

System Block Diagram:



The system architecture consists of three modular blocks:

1. Input Processing Block: Processes all external inputs including the Start and Reset signals (after debouncing) and the one IR sensor reading.
2. Control Block: Contains the main Finite State Machine (FSM) which governs the game sequence. It manages the timer enable signal, monitors rising-edge events from sensor to update score and asserts game-over conditions when score reaches 60 or when the timer expires
3. Output Processing Block: Consumes the game state and internal registers to drive all player-visible outputs. This includes the 7-segment display (score and countdown timer) and the game-over blinking LED, which blinks when the game finishes.

Input Block:

The Input Block manages all external user inputs required for the Point Blank on the Basys-3 FPGA board. It interfaces directly with the Start button, reset button, and the single IR sensor that detects when a can has been knocked down.

The mechanical slide switches (Start and Reset) and the IR input all pass through a debounce module to remove noise before reaching the Control Block. The IR sensor detects a successful hit using a rising edge (0→1 transition) after debouncing, which indicates the moment the sensor beam is interrupted

After processing, the Input Block outputs start_clean, reset_clean, and sensor_clean signals to the Control Block, where they are used to enable the timer, update score, and manage game-state transitions.

```
    output led
);
    //yahan debouoing karray to all our inouts
    wire start_clean, reset_clean, sensor_clean;
    debounce db_start(.clock(clock), .unstableinputfromsensor(start), .stable_out(start_clean));
    debounce db_reset(.clock(clock), .unstableinputfromsensor(reset), .stable_out(reset_clean));
    debounce db_sensor(.clock(clock), .unstableinputfromsensor(sensor_input), .stable_out(sensor_clean));

    //timer mod se called
```

Pin Configurations and I/O Details:

Peripheral	Basys-3 Pin	Signal Name	Description	Logic
Start	T1	start	Starts the game. Pass through the debounce module to remove switch noise.	Active High
Reset	R2	reset	Resets game logic, score, and timer. Also debounced for stable transitions.	Active High
Sensor	P18	sensor_input	Detects can's presence. Rising edge after debounce = hit event.	Active High
Clock	W5	clock	Drives debounce, FSM, timer and score logic	-

Switch and IR Sensor Processing:

For testing and verification, a simple sensor-test module was used to directly mirror the IR signal to an onboard LED. And the debounce module was used to clean all the slide switch inputs as we experienced fluctuation and noise with them in the very beginning of our project.

Project Summary | sensor_test.v

C:/Users/T490/sensor_test/sensor_test.srcs/sources_1/new/sensor_test.v

```
1  `timescale 1ns / 1ps
2  module sensor_test(
3      input clock,        // 100 MHz
4      input sensor_in,
5      output reg led
6  );
7      always @(posedge clock)
8          led <= sensor_in;  // LED mirrors sensor state
9  endmodule
10
```

Project Summary | milestone2_constraints.xdc | topmodule.v | debounce.v

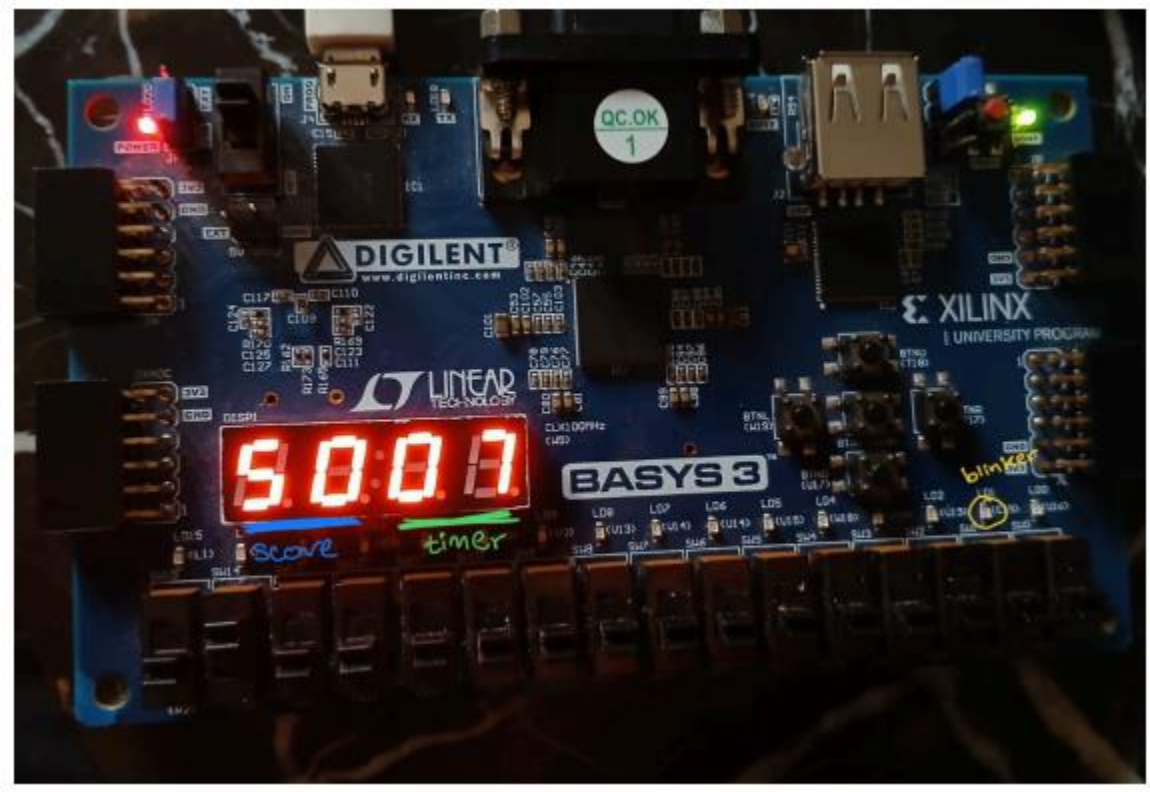
C:/Users/T490/milestone2/milestone2.srcs/sources_1/new/debounce.v

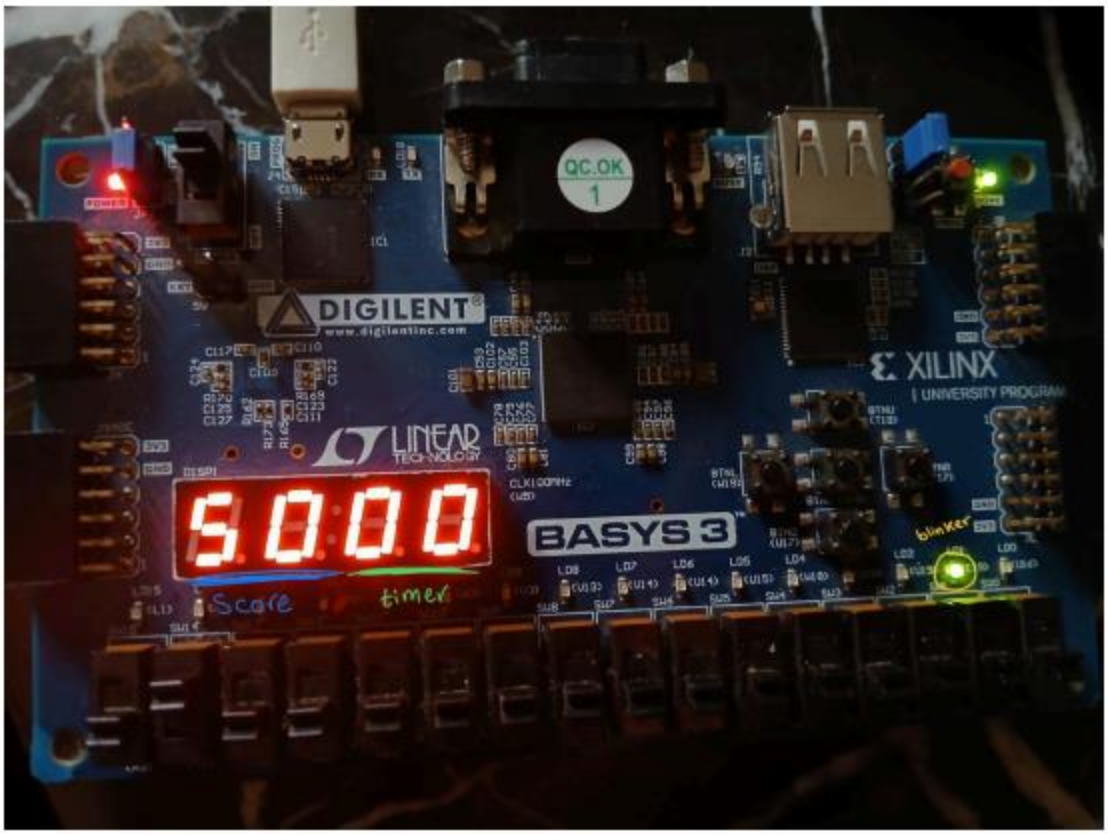
```
1  `timescale 1ns / 1ps
2  module debounce(
3      input clock,
4      input unstableinputfromsensor,
5      output reg stable_out
6  );
7      parameter max_count = 200000; // this means thatif input stays samne for 2ms, we can accept it
8      reg [17:0] count = 0; //this is to count the maxcount
9      reg prevstate = 0; //this remebers the prev 0/1
10
11     always @(posedge clock) begin
12         if (unstableinputfromsensor == prevstate) //agr the input we get is the same as prev val
13             count <= 0;
14         else begin
15             count <= count + 1; //wanra count how long this val stays same
16             if (count >= max_count) begin //agr count is >= 2ms
17                 prevstate <= unstableinputfromsensor;
18                 stable_out <= unstableinputfromsensor;
19                 count <= 0;
20             end
21         end
22     end
23 endmodule
24
```

Output Block:

The Output Block manages the three visual components relating to the game's status. The three components are; the score display, countdown timer, and a blinking LED used to indicate that the game has ended. The left two digits indicate the score; the right two indicate the timer on the four-digit segment display and an LED button for the blinker. All these outputs are based on the signals generated by the Timer and Score modules during gameplay.

The score has not reached 60, and the timer is still going, therefore the game hasnt ended and the blinker is off.





The timer ended, and the blinker got triggered indicating the end of the game.

Timer Display:

The timer display is implemented using two modules: a countdown timer and a seven-segment display.

The timer module uses a clock divider that adjusts the FPGA's 100 MHz system clock to a 1 Hz signal. This makes sure that the countdown goes down once a second. The timer begins at 30 seconds and decrements while the game is running.

The seven-segment module gets the timer value from the timer module and splits it into tens and ones and displays them on the **right two digits** of the FPGA. A refresh counter cycles through the four anodes which allows the two timer digits and two score digits to appear simultaneously even though only one digit is active at a time. Segment patterns are produced using the mappings for digits 0–9 which we learned in our Labs.

Once the countdown reaches zero or the score reaches its max, the countdown stops and triggers the blinker LED module to indicate that the game has ended.

```
    end

    always @(posedge clock) begin
        if (reset) begin // agr reset switch on hai to sab initial stage pay wapis la jao
            count <= 30;
            running <= 0;
            div <= 0;
            done <= 0;
        end
        else if (switch && !done) //agr start swicth on hai and timer khatam nhi hoa to start the timer
            running <= 1;

        if (running && !done && !stopwhenscore) begin //We only count if timer is running, khatam nhi hoa, and not stopped by score mod
            if (div == 99_999_999) begin //100 MHz (100 million ticks per second), So if you count to 99,999,999, that takes 1 second
                div <= 0; //Reset div to 0 after every second.
                if (count > 0)
                    count <= count - 1; // ye to simple count down horha
                else begin // jab timer 0 hojata to done ko 1 dedo
                    count <= 0;
                    running <= 0;
                    done <= 1;
                end
            end else
                div <= div + 1; //agr 1 sec nhi hoa to inc divider
        end
        else
            div <= 0; //reset
    end
endmodule
```

Score Display:

The score display is implemented using the score incrementing logic and the seven-segment display module. The scoring system increases the score by 10 points every time the sensor detects a rising edge.

The score module keeps track of the current score and stops incrementing once the maximum value of 60 is reached. When the score hits this limit, it outputs a game over signal which stops the timer and ends the game.

The seven-segment display receives the updated score value and splits it into tens and ones digits so they can be shown on the **left two digits** of the four-digit display. Once the game ends, whether by reaching 60 points or by the timer ending, the score display freezes and shows the final score until the system is reset.

```
    end

    always @(posedge clock) begin
        if (reset) begin //agr reset press to sab zeros
            score <= 0;
            prev_hit <= sensor_hit;
            game_over <= 0;
            locked_hits <= 0;
        end
        else if (!game_over) begin //iske nechay anything will only work if game_over is 0
            if (timer_done) //agr timer mod se done 1 aya to game over
                game_over <= 1;
            else begin
                if (running && sensor_hit && !prev_hit && locked_hits < 6) begin // agr pos edg
                //are diff and obv 60 se kam hai to score +10 and 6 mein 1 shot use hogya
                    score <= score + 10;
                    locked_hits <= locked_hits + 1;
                end
                prev_hit <= sensor_hit; //after that current state becomes prev state

                if (score >= 60) begin //bas 60 tak jaiga if you reach it game over.
                    score <= 60;
                    game_over <= 1;
                end
            end
        end
    end
endmodule
```

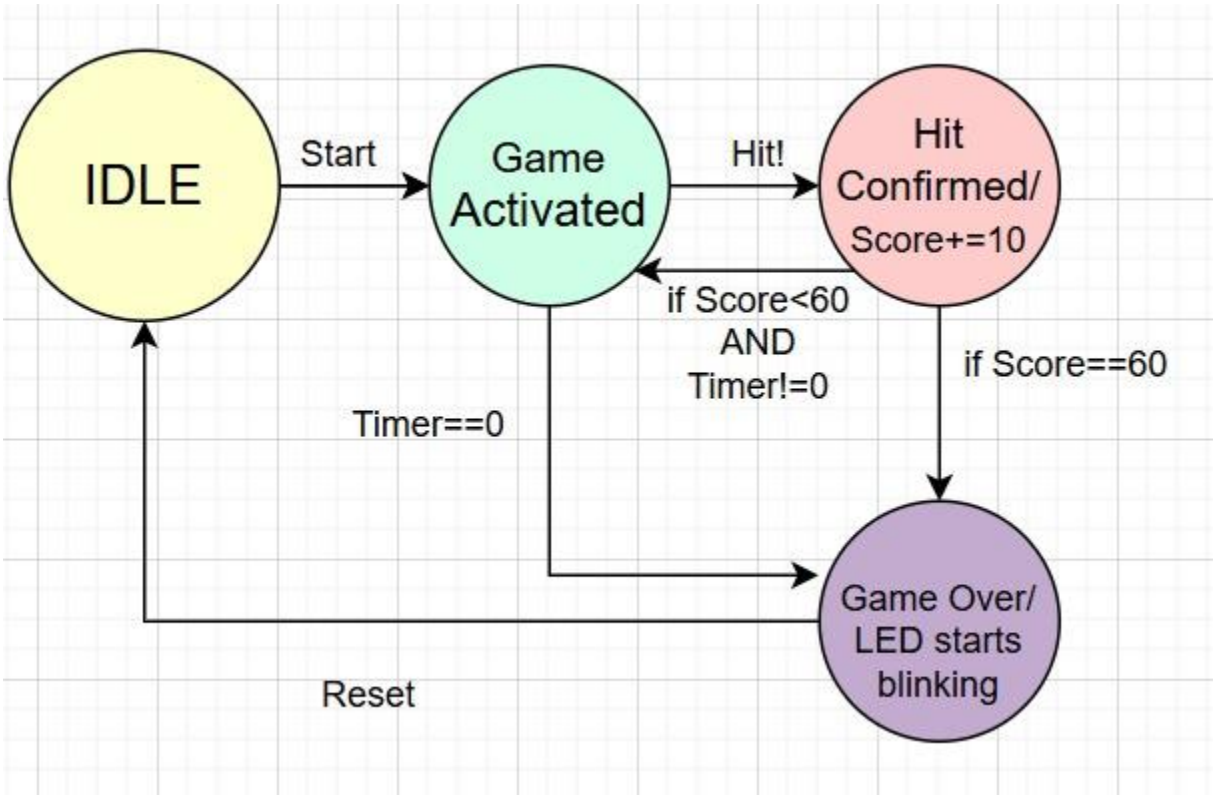
Blinking LED:

The blinker module is used to visually indicate that the game has ended. It operates by dividing the FPGA's 100 MHz system clock into a much slower blinking frequency of 1 Hz and toggles the LED state once every 0.5 seconds.

The module activates only when the game is over, which happens either when the timer reaches zero or the score reaches its maximum value of 60. Once the blinker receives this signal, it begins alternating the LED between on and off states. This blinking signals that the game is over. When the game is not running or the system is reset, the blinker is disabled, and the LED is off.

```
1  `timescale 1ns / 1ps
2  module blinking(
3      input clock,
4      input running, //this is basically the start buttn and reset button from ti
5      output reg led //led om fpga
6  );
7      reg [25:0] blink_div = 0; //every clock cycle, it increases by 1, used to s
8
9      always @(posedge clock) begin
10         if (running) begin // agr start hai to
11             blink_div <= blink_div + 1; // counter inc by 1
12             if (blink_div == 50_000_000) begin //agr ints hogya to
13                 blink_div <= 0; // reset it
14                 led <= ~led; // agr on to off, off to on
15             end
16         end else begin
17             blink_div <= 0; //disabled
18             led <= 0;
19         end
20     end
21 endmodule
22
```

Control Block:



In Milestone-2, the Control Block was implemented without a formal Finite State Machine (FSM). At this stage of the project, we had not yet covered FSM theory in class, and so we did not use explicit state variables, state registers, or next-state logic in the Verilog implementation. However, even without defining states, the internal signals generated by the modules naturally create an FSM behavior that sequences the game correctly.

The system relies on three key control signals:

- Running -> set by the Start button; enables the timer and activates gameplay
- done -> raised by the Timer module when the countdown reaches zero
- game_over -> raised by the Score module when the score reaches 60

These signals act like the conditions of an implicit FSM:

- When running = 0, the system behaves like an idle state.
- When running = 1, the system behaves like a game active state.

- A valid hit (detected via a rising-edge transition of the debounced IR sensor signal) triggers scoring.
- When done = 1 or game_over = 1, the system enters a game-finished state where the score and timer freeze, and the blinking LED is enabled.

Thus, even though no formal FSM is coded using case statements or state registers, the interaction of these control signals successfully sequences the entire game.

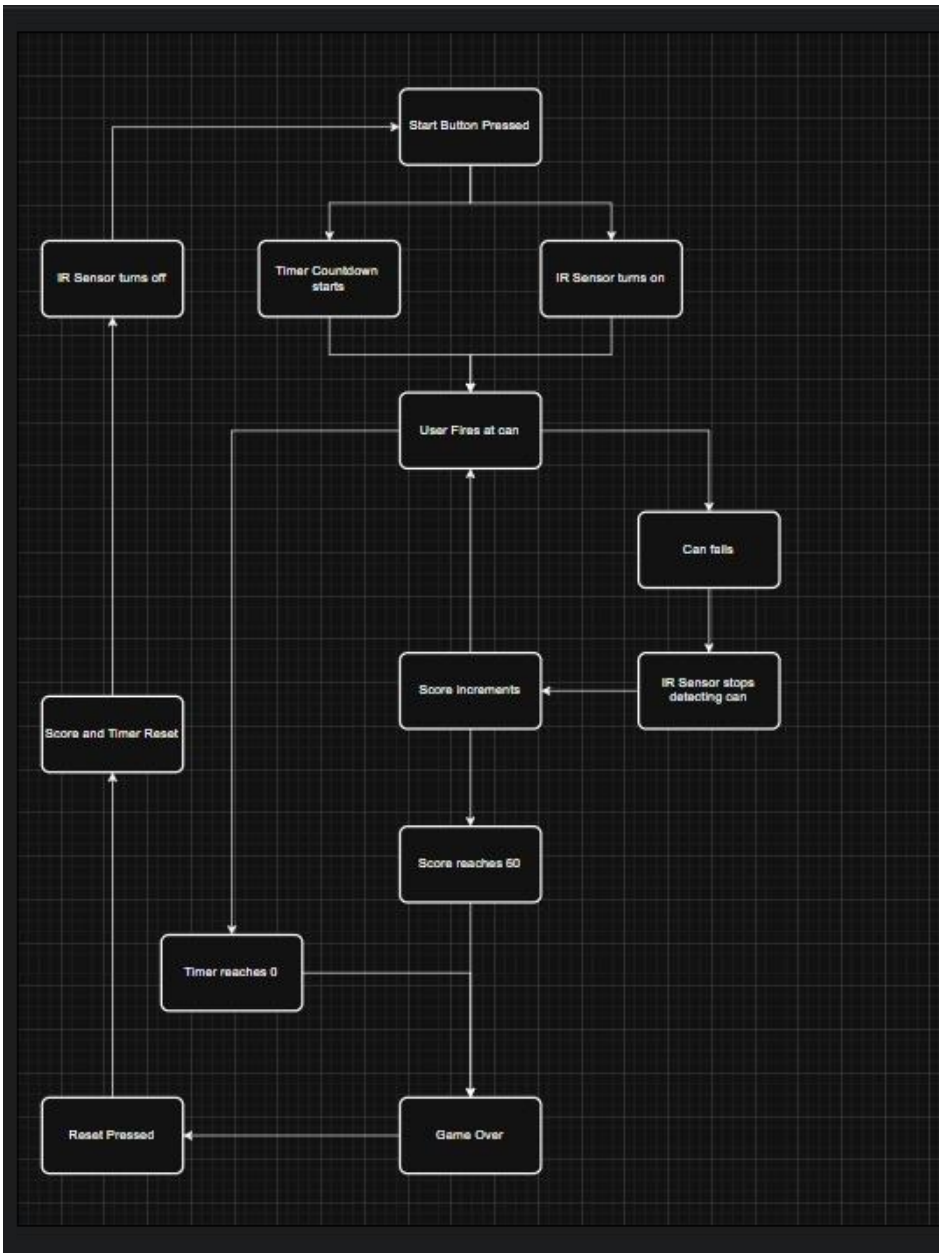
For the final milestone, the same behavior will be re-implemented using a proper, fully defined FSM with named states (IDLE, GAME_ACTIVE, HIT, GAME_OVER). Milestone-2 therefore represents the signal-driven version of the control logic, while Milestone-3 will convert this into an explicit FSM-based implementation.

User Flow Diagram:

The user flow outlines the complete gameplay sequence from system startup to game over, focusing on the Start/Reset inputs, debounced IR-based hit detection, real-time scoring, and the 30-second countdown.

The interaction follows a clear pathway: User Inputs → Input Block → Control Block → Output Block. When the player presses Start, the debounced signal enables the timer and activates gameplay. During the active phase, the system continuously monitors the IR sensor, where a rising-edge (0→1) transition after debouncing indicates that the target has been hit. Each valid hit increments the score by 10 points, and both the score and timer are displayed simultaneously through the multiplexed seven-segment display. Gameplay continues as long as the timer has not reached zero, and the score has not reached the maximum of 60.

When either condition is met, the Control Block asserts the game-over signal, freezing the outputs and activating a blinking LED to indicate the end of the round. Pressing the Reset button returns all modules to their initial state, completing the flow and allowing a new session to begin.


Start Button Pressed
IR Sensor turns off
Timer Countdown starts
IR Sensor turns on
User Fires at can
Can falls
Score increments
IR Sensor stops detecting can
Score and Timer Reset
Score reaches 60
Timer reaches 0
Reset Pressed
Game Over